Free Open-Source Engine Turns AI Agent Workflows Into Verified Contracts

circuit, from Punt Labs, replaces prompt-based process discipline with small, model-checked state machines. The agent may request progress; the machine decides whether progress is valid.

"Tests pass" means the gate ran the team's own command, one that predates the session or carries a reviewed pin, and recorded the exit status.

Press Release

SAN FRANCISCO, March 1, 2027: Punt Labs today announced circuit, a free, open-source workflow engine for engineering teams that run AI coding agents daily on repositories already guarded by CI. Those teams watch every run today: they have the gate discipline to leave a loop alone, but no account of what happens inside it beyond the agent's own report. Teams write a workflow such as test-driven development as a small formal state machine and prove it sound before first use. At runtime the machine, never the agent, decides when the workflow advances. Instruction files, skills, and hooks advise an agent and hope it complies. circuit enforces: each step is gated on evidence from commands such as the test suite, the linter, and CI status (see FAQ 2). Teams get the account that earns the permission to stop watching: a loop that refuses to advance on an unmet condition, and a trace that proves the process was followed.

Problem

Engineering teams that adopt AI coding agents encode their process in prose. Instruction files, skills, hooks, and ever-longer prompts say "write the failing test first," "never merge until review is clean," "run the full gate before committing." Agents follow these instructions most of the time, and drift the rest of the time. They skip the failing-test step, declare success while the build is red, and treat their own summary as proof the work is done. Each drift costs a review cycle to catch and a rework cycle to fix. It also caps autonomy, because a loop that cannot be trusted must be watched. Current fixes sit at two extremes. More prose makes the context longer and the compliance no more certain; every soft mechanism relies on the agent choosing to obey. Governance platforms add identity, delegation contracts, and audit trails. The team adopts an entire organizational model to get one guarantee: the workflow is followed.

Solution

With circuit, a team writes the workflow once, as a state machine of a few dozen lines: named states, allowed transitions, and the facts each transition requires. During development, a model checker verifies the machine, so an unreachable state or a rule conflict is caught before an agent ever runs the loop. At runtime, a single small binary interprets the machine with no server, no cloud, and no verification tooling required. Each gating fact is bound to a command. A transition that requires passing tests runs the test suite; one that requires clean review reads the review state itself. When an agent asks to advance, circuit advances or refuses, naming exactly which condition failed. The refusal lands in the agent's context as its next instruction. In drive mode, circuit runs the loop itself. It prompts the agent with the current state and valid next steps, and validates every requested transition (see FAQ 4). It also writes a step-by-step trace, so the team can read afterward what the agent did in each state and why each gate opened.

Customer Quote

> *"We ran agents every day and watched every one of them. CI catches a bad merge, and nothing catches an agent that reports itself done. We had a beautifully written TDD instruction file, and our agents still wrote the implementation first whenever the test was awkward. With circuit the implement step simply refuses until a failing test exists, and the agent routes around its own laziness instead of ours. For the first two weeks I still opened the terminal every twenty minutes. Every time, the loop was either working or parked on a refusal it had already named. Last month I started a quality-ratchet loop on our pi driver, in a throwaway worktree, before a long weekend. On Tuesday I read the traces and merged nine refactors, each with the gate that had opened for it. I stopped checking."*
>
> — **Maya Okafor**, Staff Engineer at Fieldline Robotics

Getting Started

Getting started takes three steps and no infrastructure. First, install the single binary and run `circuit list` to see the shipped machines, including test-driven development and a pull-request watch loop. Second, run `circuit scaffold` in a repository. The scaffold generates stub bindings that connect the machine's facts to that project's own commands. A stub defaults to false, so an unfinished integration blocks safely instead of passing silently. Third, pick the integration mode. In drive mode, `circuit drive` owns the loop: circuit prompts the agent, validates every requested transition, and runs to a terminal state with nobody watching. Drive mode works with pi today. In tool-call mode, the harness owns the loop and calls circuit at each step. The gates still refuse every invalid transition. Whether the loop runs at all depends on the harness making the call, so tool-call mode alone does not earn the permission to stop watching. Claude Code, opencode, and any coding harness that can run a command-line tool get tool-call mode today. Either way, a team can watch a machine refuse an invalid transition within ten minutes of installing, with no server and no account.

Spokesperson Quote

> "Everyone is trying to fix agent process drift with better prose, and prose is advice. The agent grades its own homework. Our design bet was to take the workflow out of the prompt entirely and put it in a tiny formal machine. Prove it correct at development time, keep the runtime small and boring, and gate every step on a command the agent cannot fake. The pattern is decades old; safety-critical rail systems have run on proven state machines since the nineties. What surprised us in our own use was the behavior change. When the quality gate refused to open, the agent stopped arguing it was done and went back to refactoring until the gate passed. Enforcement turned out to be better feedback than instruction."
>
> — **Jim Freeman**, Founder at Punt Labs

Call to Action

circuit is available today at `https://github.com/punt-labs/circuit`, free under the MIT license. It ships as one Go binary with five ready-to-run workflow machines. Any agent harness that can call a command-line tool runs circuit in tool-call mode today, gates included. Drive mode,

where circuit owns the loop and runs it with nobody watching, works with pi today. Adapters for Claude Code and opencode follow. Start with the ten-minute walkthrough in the repository README.

Frequently Asked Questions

External FAQs

Q1: What is circuit and who is it for?

circuit is a free workflow engine that makes an AI coding agent follow a defined process. It is for engineering teams that run agents such as Claude Code, Cursor, or pi daily, on repositories where CI gates already decide what merges. Those teams supervise every agent run today, because gate discipline covers the merge and leaves the loop that reaches it unaccounted for. The team writes the workflow as a small state machine, and circuit refuses any step whose required evidence is missing. The core job: give a supervising team the evidence it needs to stop watching, without asking anyone to trust the agent's own report of success.

Two integration modes exist, and they deliver different amounts of that promise. In drive mode, circuit owns the loop, prompts the agent, and validates every requested transition. The unattended guarantee holds end to end there, and pi is the harness with drive mode today. In tool-call mode, the harness owns the loop and calls circuit at each step. Gates still refuse invalid transitions there, and compliance with the loop depends on the harness calling at all. Tool-call mode alone therefore leaves a supervising team something to watch. Claude Code and opencode run in tool-call mode today, and the adapters in FAQ 20 deepen both.

Q2: Why are skills, instruction files, and hooks not enough?

Prose instructions depend on the agent choosing to comply, and measurement shows compliance drifts. Across 20,574 coding-agent sessions in the wild, developer constraint violation was the largest misalignment symptom at 38.33%, and instruction-following failure was the largest root cause at 36.49% [1]. Better prompting narrows the gap without closing it: the failure pattern is architectural, not a wording defect [2]. Hooks intercept single tool calls, so they cannot see whether a multi-step workflow ran in order. circuit sits outside the model. A transition either satisfies its guard or it does not, and no phrasing changes the answer.

Q3: How is circuit different from LangGraph?

LangGraph compiles an agent workflow into a state graph and validates the graph's structure before execution [3]. Its transitions are still decided by model outputs, so the system being constrained also judges the constraint. circuit decides transitions from externally executed commands: the test suite, the linter, CI status. circuit is engine-shaped: you keep your existing harness and add one binary; you do not rewrite your agent as a graph.

Q4: Why B-Method state machines instead of a YAML workflow file?

A YAML file can list states and transitions; it cannot tell you the workflow is sound. B machines are model-checked during development, so deadlocks, unreachable states, and rule conflicts surface before an agent ever runs the loop. The method has industrial history: the automatic train control for Paris Métro Line 14 was developed in B, with no bug found in over 25 years of revenue service [4]. Amazon reported comparable payoff from formal specification of AWS designs [5]. The strict Circuit-B profile is the public contract, and it stays the artifact a team versions and reviews. Layers above it, such as an agent drafting a machine from a plain-language description or a compiled front end, emit that same profile (FAQ 5). Nothing above the profile changes what ProB verifies or what the runtime interprets. Runtime stays cheap: a small

Go interpreter reads the machine, and the verification tooling is a development dependency only.

One platform limit applies, and it costs machine authors time on one platform. ProB publishes universal macOS binaries, so Apple Silicon verifies locally. It publishes no Linux aarch64 build. On ARM Linux the design-time check runs in CI or an x86_64 container, and each edit-and-verify cycle waits on a container start or a CI run. Nothing about running a finished machine changes on any platform. The runtime binary runs anywhere Go does.

Q5: Do I need to know B or formal methods to use circuit?

Not to use the shipped machines. `circuit list` shows them; `circuit scaffold` wires one to your repository's commands; you never open the machine file. Authoring a new machine means writing a few dozen lines in the strict Circuit-B profile, about the effort of writing a Makefile. That comparison is our own estimate, untested outside Punt Labs, and the timed exercise in FAQ 17 runs it against an agent-assisted attempt. Unsupported constructs fail with a diagnostic that points at the offending line.

The intended authoring loop for a team's sixth machine runs through an agent. A team describes the workflow in plain language, and the agent drafts the machine using the five shipped machines as patterns. ProB verifies that draft exactly as it verifies the shipped five, so a drafting error fails the model check at design time. A human then reads the machine and approves it before it gates anything, the same review any other committed file gets. The strict profile is what the agent emits and what the runtime reads, so an agent-drafted machine takes no separate path through the verifier. Agent-authored check bindings are a different question, and FAQ 18 answers that one.

Q6: How do I get started?

Install the single binary. Run `circuit list` to pick a machine such as tdd-flow or pr-watch. Run `circuit scaffold` to generate check bindings for your repository, then start the loop from your harness or with `circuit drive`. The first refused transition, with its named failing condition, appears within about ten minutes.

Q7: How much does it cost?

Nothing. circuit is MIT-licensed open source with no paid tier. You pay only for the agent and model you already run.

Q8: What happens to us if Punt Labs stops working on circuit?

Your machines, bindings, and traces are text files in your own repository. The binary is MIT-licensed and runs with no server and no telemetry. Nothing you depend on requires us to stay online, or to stay in business. An abandoned binary keeps gating correctly the day after abandonment. What stops is fixes, new machines, and adapter updates, and the license lets anyone fork and continue those. Support today is GitHub issues from a small team, with no service commitment and no support contract on offer.

Q9: What happens to my code and data?

circuit runs entirely on your machine. There is no server, no account, and no telemetry. Session state lives in JSON files inside your repository's temporary directory, and traces stay local unless you choose to share them.

Q10: What happens when a gate is wrong, or an emergency requires bypassing the process?

Stop the session and act by hand; circuit constrains the agent's loop, never your shell. The durable fix is an edit to the machine or its bindings, which are text files that travel through code review like any other change. The workflow itself becomes versioned and reviewable.

Q11: What does a false refusal cost, and how do we keep gates honest?

A wrong gate blocks valid work, and a process tool that blocks valid work gets removed. That abandonment path is the most plausible long-term failure, so it gets its own instrumentation. Every refusal names the failed condition and the command behind it, so diagnosis is one trace line. The repair is a small reviewed edit to the machine or a binding. We track override rate next to interception rate (FAQ 24): sessions stopped by hand, and gates loosened to get past a refusal. A rising override rate means the machine is refusing valid work, and we treat that as a defect with the same urgency as a red build.

Internal FAQs

Value & Market

Q12: What is the total addressable market?

No survey measures demand for an agent-workflow enforcement layer, so this answer reports only what the adjacent data supports. By January 2026, 90% of surveyed developers used an AI tool at work [6]. By mid-2026, 68% used an AI agent daily [7], while only 29% said they trust AI output to be accurate [8]. Those two figures describe one population: developers who run an agent every day and do not trust what it reports, so they watch it. Our serviceable segment sits inside that population: teams running agents against CI on repositories they must keep healthy, gate discipline in place, supervision still in place.

The size of that segment is unmeasured. Two filters narrow the surveyed population, and neither carries a number we can defend. The first is gate discipline: what share of teams enforce a merge gate strictly enough for a refusal to mean anything is contested, and no survey reports it. The second has no figure either: some gate-disciplined teams already leave agent loops running unwatched, and the rest supervise every run. The split decides which pitch lands. An already-unattended team adopts circuit for a refusal and a trace it can audit afterward. A supervising team adopts it for the permission to stop watching, and converts only once the trace answers what watching answered. A headline segment count stacked on two unmeasured filters would compound the guesswork, so we report none.

The recruiting funnel in FAQ 17 is the measurement. Contacting teams until ten qualify yields a screening ratio and the split between the two segments, both observed, not assumed. Whatever that funnel reports is a floor, and format gravity is the reason (FAQ 14). If the machine format and the trace format become what every harness assumes it will find, the addressable pool grows toward the full agent-daily population instead of this pre-filtered subset. The discipline runs the other way too, with one falsifiable trigger. If screening fifty teams cannot produce ten qualifying interviews, the segment as defined is too thin to build for, and the definition itself gets rethought before any further investment.

Q13: What evidence do we have that customers want this?

Primary customer evidence does not exist yet. The only customer to date is Punt Labs itself, which we flag as the classic high-risk signal. The problem itself is well measured (FAQ 2). Our dogfood results: circuit-driven TDD slices delivered merged production changes; a failing quality gate forced repeated refactor loops until it passed; a pr-watch machine gated a live pull request through fix, review, and merge. In the wild, visible resolution of agent misalignment usually requires explicit developer pushback [1]. circuit automates the subset of that pushback expressible as a command's exit status. The mechanism claim rests on inference plus our own dogfood observation. No controlled study yet compares externally gated agent work against self-assessed agent work. Running that comparison on our published traces is part of the validation plan in FAQ 17.

Q14: Who are the competitors, and why will we win?

Sponsio is the closest product: it compiles policy into deterministic finite-state contracts checked at every tool call, with no model in the enforcement path [9]. Its formal layer is LTL automata for general tool-call governance; its public materials describe no design-time model checking, and it does not target coding workflows such as TDD or PR cycles. LangGraph owns mindshare for

graph-shaped orchestration, and its transitions are judged by model outputs [3]. One research prototype model-checks an agent runtime in TLA+ [10], an early signal of the direction. Any of these players could copy the pattern within a quarter, and we are playing for that outcome. This is an open-standard race, and the prize is convention: the machine format, the trace format, and the check-registry pattern becoming what every harness assumes it will find. If Sponsio ships TDD and PR-cycle contract templates next quarter, it ratifies the category; circuit is the implementation already running those loops in production. Our ground while that race runs: a formal method proven in rail service [11], model checking required at design time, checks bound to repository-authored commands, and neutrality across agent harnesses (see Feature 16).

Distribution velocity is the strategy, so the capacity behind it belongs in the same answer. Shipping working machines for the loops teams run every day depends on authoring throughput. FAQ 21 names authoring as the first thing that breaks at scale, and FAQ 23 puts the budget at a few hours a week from one owner. We therefore test authoring throughput instead of assuming it: the dogfood month in FAQ 17 counts machines authored and hours spent per machine. A rate that cannot produce a sixth shipped machine in that month sends templates and diagnostics ahead of new machines.

The MIT license is irreversible. Every published version stays MIT, so no later pivot can reclaim the format from the teams and tools that adopt it. That permanence is what makes the format safe to standardize on. Publication came first by design: the runtime, the five machines, and the public repository exist now, and the interview month gates further investment, not the first release. Publishing first puts a running implementation in the race while the convention is still unsettled.

Q15: How does circuit relate to ethos and beadle?

They are complementary layers, and ethos is the heavier one by design. ethos answers who may do what: identity, delegation contracts, write-sets, audit. circuit answers when work may advance: a process gate with no organizational model attached. A team can adopt circuit alone in an afternoon. Internally the integration runs the other way. ethos mission pipelines are informal state machines today and can execute on circuit machines. beadle's planned autonomous mail daemon needs exactly circuit's rule, advance only when trust checks pass, before it acts on a message. One engine, three surfaces.

Q16: What is the customer acquisition strategy?

Distribution follows the harness ecosystems, one at a time. A pi extension exists today, and pi is where drive mode runs. Claude Code is the next adapter, because the widest audience already works there, and opencode follows on the same path. Each adapter is a deep per-harness investment by choice (FAQ 26), so reach arrives one ecosystem at a time. Until an adapter brings drive mode to an ecosystem, circuit runs there in tool-call mode. The pitch there is the refusal and the trace, not unattended operation (FAQ 1). The shipped machines double as demos. Traces make sharable evidence; a blocked-transition log is a better pitch than a benchmark. Punt Labs' own tools are the first embedders (FAQ 15), so ethos and beadle installs carry circuit's pattern with them. Channels, costs, and conversion rates remain unvalidated assumptions.

Q17: What is your next step to validate the vision?

Interview ten engineering teams whose repositories already gate merges on CI. Screen for both segments: teams that already leave agent loops running unwatched, and teams that run agents daily and supervise every run. Count the screening itself. How many teams we contact to reach ten, and how they divide between the two segments, is the first measurement of the segment size left unmeasured in FAQ 12. From the already-unattended teams, count how many have built ad-hoc gating scripts around their agents; an existing workaround a team maintains is the strongest demand signal available. From the supervising teams, ask one question: what would you need to see before you stopped watching a run? An answer circuit already produces is a conversion path, and an answer no command can supply kills the framing. Each interview also carries a timed authoring exercise on a workflow the team names. The engineer attempts the machine twice: once unassisted from the profile documentation and the shipped machines, once with their own agent drafting it for review. We record both times, whether each attempt passes the model check, and what the reviewer changed in the agent's draft. The Makefile claim in FAQ 5 survives only if the unassisted attempt finishes and verifies. The assisted attempt tests the authoring loop we intend teams to use, and a draft the reviewer cannot follow fails it as surely as a draft ProB rejects. In parallel, run pr-watch and tdd-flow on every Punt Labs repository for one month and publish the traces, including every refused transition. If no team builds workarounds, no supervising team names a condition a gate can supply, and our own traces show no interceptions, the value hypothesis fails and we stop. The dogfood month also logs every override, because a team that adopts the tool and then disables it is the third failure mode (FAQ 11). None of this gates the release: the repository is public and MIT-licensed now, and publishing first is deliberate (FAQ 14). What the interviews and the traces gate is further investment. A month with no workaround-builders, no interceptions, and no sixth authored machine stops the adapter and embedding work in FAQ 20. The published engine stays MIT either way.

Technical

Q18: What are the major technical risks?

Check independence is the sharpest risk. A gate is only as trustworthy as the independence of what it runs. In one study of LLM-based test generators, up to 68.1% of one tool's final suites on the Refactory dataset passed on incorrect implementations. Broader averages ran near 26 to 35% [12]. Two provenance questions sit behind that finding, and circuit answers each with its own rule. Binding provenance asks which command a machine fact resolves to. The registry answers it: bindings resolve to repository-authored commands, and no binding is added or rewired mid-session. Check-content provenance asks what that command runs, and a machine-checked guard is the designed answer, specified and not yet built. As designed, circuit records the repository head when a driven session opens. Before a check counts toward a transition, the engine reads the history of that check's own source files. A check file with commits after the session opened fails the guard. The one way past it is a binding pinned to a commit hash the operator reviewed before the session started. A failed provenance guard refuses the transition the same way a failing test does, so an agent that edits its own gate parks the loop instead of opening it. Until the guard ships, the mid-session registry rule and merge review carry this risk alone. Content quality is a third question, and neither rule answers it. The failure rates above stand as the bound on what a check asserts, and merge review is where a weak assertion gets caught.

tdd-flow shows where that boundary falls. The failing test the agent writes during the run sits inside the session boundary by design. The gate therefore proves the sequence: the test ran, it failed, and then it passed. What the test asserts is what the human reviews at merge. An agent-drafted machine (FAQ 5) sits on the reviewed side of the same line, since it passes the model check and a human reviewer before any session starts.

Second, profile expressiveness: a strict B subset may prove too narrow for messy workflows. Mitigation: guards combine and negate externally observed facts, and anything beyond the profile lands in a reviewed check command, not the machine language. Third, harness churn: adapters track moving surfaces such as pi's RPC mode. Mitigation: adapters stay thin, the CLI is the contract, and the engine itself has no harness dependency. Fourth, unattended operation has safety requirements circuit only partly meets. A loop nobody is watching needs a spend ceiling, stall detection, and a bounded blast radius. Today a driven session stops on its own at a terminal state (Feature 4), and every transition and refusal lands in a trace the team reads afterward (Feature 5). A gate that cannot be evaluated blocks instead of opening, so a broken check parks the loop instead of releasing it. Today circuit also sets no token or dollar ceiling on a run, detects no stalled agent, and constrains no command the agent runs between transitions. A per-session budget and a no-progress timeout are roadmap items. Worktree preparation (Feature 12) is the intended blast-radius answer, since a loop confined to its own worktree cannot reach the main checkout. Until those land, the unattended claim covers process compliance, and spend limits and containment stay with the operator.

Q19: What dependencies exist, and what is the fallback?

ProB is a development and release dependency only, pinned at 1.15.1 through Nix; runtime needs nothing beyond the Go binary. ProB publishes no Linux aarch64 build, so machine verification runs on x86_64 in CI. Drive mode currently depends on pi's RPC mode, so pi is the one harness where the unattended path runs today. The hosting relationship works everywhere: any coding harness that can run a command-line tool hosts circuit in tool-call mode, with gates intact and the loop still its own (FAQ 1).

Q20: What is the delivery order, and what gets cut first?

Delivery order, without dates. Evidence runs first. The interviews and the dogfood month in FAQ 17 run against the CLI path that ships today, so validation waits on no new code. Remaining engine work runs in parallel with them: hardening the five shipped machines and making per-run prompt overrides first-class input (Feature 8). That scope already runs at low feasibility risk, so it no longer sets the pace. Second, embed circuit in ethos mission pipelines (FAQ 15). That comes early because the internal-adoption metric in FAQ 24 checkpoints it one quarter after the engine reaches its Must Do bar. Third, the harness adapters: Claude Code, where the widest audience already works, then opencode. Each adapter is a deep per-harness integration by choice (FAQ 26). Fourth, the beadle embedding and an external beta with the interviewed teams.

If scope compresses, the external beta and the beadle embedding cut first, then the opencode adapter. The ethos embedding is cuttable at a stated price. Cutting it forfeits the internal-adoption checkpoint in FAQ 24, and a forfeited checkpoint carries the same consequence as a missed one (FAQ 22). The engine and the model-check gate never cut, and the asymmetry is the reason. Weak demand is recoverable: the interviews report it early, we stop investing, and every published version keeps working. A machine that opens a gate which never held breaks

the one guarantee circuit sells, in public, on someone else's repository. A trust product does not come back from that.

Q21: What breaks first if this succeeds?

The engine breaks last; interpreting a machine of a few dozen lines is negligible work. Authoring breaks first: at many machines per organization, writing B by hand needs templates, better diagnostics, and the agent-drafting loop in FAQ 5. Every machine still needs a human reviewer, so reviewer attention becomes the ceiling once drafting gets cheap. That same limit caps the distribution velocity in FAQ 14. Machines ship no faster than one owner, at the capacity in FAQ 23, can author and verify them. Check registries sprawl second: per-repository command bindings will drift and want linting. Session visibility is third: JSON files per repository serve one team well and stop short of any fleet-level view. Each is tooling work, and none threatens the core design.

Business

Q22: What is the revenue model?

None today, and the reason is format gravity. Whatever gets built on a default format is worth charging for later. A format nobody has adopted yet has nothing to sell, at any price. So circuit gives the format away until every agent harness assumes it, which is the open-standard race in FAQ 14. The free MIT posture matches ethos and beadle, and it is the cheapest credible proof that our agent tooling holds itself to verifiable process. Costs are maintenance attention only, since there is no infrastructure to operate. A commercial layer above the engine remains undecided. Fleet visibility and hosted machine authoring are the candidates, and neither is designed or priced. The internal-adoption metric in FAQ 24 puts a checkpoint and a consequence on the strategic-value claim above.

Q23: What does circuit cost us to run?

Maintenance attention, made explicit. The owner is Punt Labs' founder, with agent tooling carrying routine upkeep. The assumed steady-state load is a few hours a week outside integration pushes, an order-of-magnitude guess we will replace with observed effort. That few hours a week is also the authoring budget behind the velocity claim in FAQ 14. Authoring a sixth machine during the dogfood month is the throughput test. Support arrives as GitHub issues with no service commitment, and the no-telemetry choice (Feature 17) means we never see the failures nobody reports. The displaced alternative is time on ethos and beadle, which the embeddings in FAQ 20 partially return.

Q24: What are the key metrics, and what makes us reconsider?

Five metrics, each with a decision threshold. Interception rate: refused transitions per driven session, read from traces. If a month of dogfood shows near-zero interceptions, the layer guards against a problem we do not have, and we reassess. Interceptions alone cannot prove health, since a machine that refuses everything scores perfectly, so the metric pairs with override rate. Override rate: sessions stopped by hand or gates loosened to get past a refusal (FAQ 11). If overrides exceed one in five refusals, a starting threshold we will recalibrate against observed runs, the gates are wrong, and machine fixes come before new machines. Unattended completion: driven loops reaching a terminal state without human intervention, targeting a majority of runs,

since babysitting is the cost we set out to remove. External embedding: repositories outside Punt Labs with a committed machine file. If that count is still zero two quarters after the Claude Code adapter ships, the open-source bet needs rework. Internal adoption: an ethos or beadle production pipeline running on circuit machines, with ethos mission pipelines the named first embedding (FAQ 15). The checkpoint is one quarter after the engine reaches its Must Do bar. If no internal pipeline runs on circuit machines by then, the strategic-value justification in FAQ 22 is retired, and no new adapter work starts until one internal pipeline ships. The free posture then stands or falls on open-standard grounds alone (FAQ 14).

Q25: Why now?

Agent adoption climbs while agent trust falls (FAQ 12). 68% of developers use an agent daily, and trust in AI output stood at 29%, the lowest the Stack Overflow survey has recorded [7, 8]. Those two numbers describe the same teams: agents run every day and someone watches every run, because nothing outside the agent confirms what it reports. A controlled study found experienced developers roughly 19% slower with AI assistance while believing themselves faster [13], so self-perception is unreliable on both sides of the keyboard. Enforcement architectures are emerging in research and product [10, 9], and none yet owns coding-workflow discipline. The window is the gap between agents everywhere and guarantees nowhere.

Q26: What are we not building?

No scheduler: the harness or the operator owns cadence. No GitHub client and no persistence layer inside the engine; observations arrive through bound check commands, and a direct integration would couple a tiny engine to moving external APIs. No MCP server either, and that is a strategy call, not a coupling call. Two integration relationships define the product: circuit drives the coding harness, or the harness (or an embedding product such as ethos) hosts circuit. Both reach for capability specific to one coding harness, capability a common protocol surface would leave on the table. Native MCP support is absent from pi altogether, and opencode exposes a better integration path than Claude Code for some capabilities. No general B interpreter: the strict profile is the public contract, and authoring layers sit above it without widening it. No model in the enforcement path, ever. No hosted service and no telemetry. Each exclusion keeps the runtime small enough to trust and to review.

Risk Assessment

Risk	Rating	Assessment
Value	HIGH	Two reasons. Demand for an enforcement layer is unmeasured, and the only customer so far is Punt Labs itself, though the problem itself is externally measured. The moat is a speed-to-convention bet, with nothing structural behind it and authoring throughput the tested limit on that speed (FAQ 14). The interviews in FAQ 17 test the demand side directly.
Usability	MEDIUM	Using a shipped machine is a three-step start. Authoring a new machine needs B literacy or an agent drafting it for human review (FAQ 5), and we have not tested that loop outside Punt Labs. No analogous product gives customers a mental model.
Feasibility	MEDIUM	The engine, five machines, drive mode, traces, and CI gates run today, and that scope is low risk. The announced product also needs the unbuilt Claude Code adapter, drive beyond pi, and the session-boundary provenance guard for checks (FAQ 18).
Viability	MEDIUM	Free open source with no operating cost. Maintenance attention is owned and estimated in FAQ 23. The rating stays MEDIUM until the internal-adoption metric in FAQ 24 reports. The strategic-value case for a free engine rests on an internal embedding that has not happened yet (FAQ 22).

Feature Appendix

Must Do

- **F1.** **Circuit-B engine**: a strict, multi-pass parser and evaluator with diagnostics anchored to the author's source; the machine is the sole authority on valid progress
- **F2.** **Model-check gate**: every shipped machine passes ProB initialization and model checking before release
- **F3.** **Check bindings**: machine facts bound to repository-authored commands; scaffolded stubs default to false so an incomplete integration blocks safely; the designed session-boundary provenance guard (FAQ 18) refuses a transition when a check's source file carries commits made after the session opened, unless the binding pins a commit hash reviewed beforehand
- **F4.** **Multi-session runtime**: suspend and resume across short CLI invocations, independent sessions per machine, auto-stop on terminal states
- **F5.** **Drive mode with traces**: circuit owns the loop, prompts the agent per state, validates every requested transition, and writes a step-by-step trace
- **F6.** **Shipped machines**: tdd-flow, pr-watch, review-flow, retry-flow, and build-job as ready-to-run workflow contracts

Should Do

- **F7.** **Claude Code adapter**: native commands and per-turn context injection for the largest audience, with the opencode adapter next on the same per-harness path (FAQ 16)
- **F8.** **Per-run prompt overrides**: named prompt presets as run input rather than worktree file edits, unblocking ratchet-style runs
- **F9.** **ethos embedding**: mission pipelines execute on circuit machines instead of informal stage logic
- **F10.** **beadle embedding**: the autonomous mail daemon advances only through trust-gated transitions
- **F11.** **Richer traces**: capture check command output, machine-readable refusal reasons, and each check file's author and last-modified commit alongside transitions
- **F12.** **Worktree preparation tooling**: one command yields a ready worktree, so a gate failure reflects agent work rather than environment setup

Won't Do

- **F13.** **Scheduler**: cadence belongs to the harness or the operator; a built-in scheduler grows circuit into the platform it refuses to be
- **F14.** **Built-in GitHub, MCP, or persistence integrations**: GitHub clients and persistence stay out on coupling grounds, since observations arrive through bound check commands and a direct integration would tie a tiny engine to moving external APIs; MCP stays out on strategy, because deep per-harness adapters reach capability that a common protocol surface never exposes (FAQ 26)
- **F15.** **Full B-Method support**: general B belongs to ProB, and the strict profile stays small enough to keep the evaluator reviewable and testable; friendlier layers above the profile are in scope, while widening the profile is not
- **F16.** **Any model in the enforcement path**: a language model judging transitions reintroduces the self-report problem circuit exists to remove

F17. **Hosted service or telemetry**: trust in an enforcement tool starts with it running entirely on the team's own machines

References

[1] Ningzhi Tang et al. *How Coding Agents Fail Their Users: A Large-Scale Analysis of Developer-Agent Misalignment in 20,574 Real-World Sessions*. Observational study of 20,574 real coding-agent sessions across 1,639 repositories (Cursor, GitHub Copilot, Claude Code, Codex, OpenCode, Gemini CLI); quantifies instruction-following failure, developer constraint violation, and inaccurate self-reporting as recurring misalignment symptoms. 2026. URL: https://arxiv.org/abs/2605.29442.

[2] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. *Why Do Multi-Agent LLM Systems Fail?* Taxonomy of 14 multi-agent LLM failure modes (MAST) from 1,642 execution traces across 7 frameworks; argues failures are architectural, not solvable by base-model improvement alone. 2025. URL: https://arxiv.org/abs/2503.13657.

[3] LangChain. *LangGraph Overview*. A compiled LangGraph StateGraph is a finite state machine whose transitions are driven by LLM outputs and tool results; graph compilation validates connections and detects cycles before execution, but transition validity is decided by the same LLM outputs being constrained. 2026. URL: https://docs.langchain.com/oss/python/langgraph/overview (visited on 08/30/2026).

[4] CLEARSY. *Extension of Line 14 of the Paris Metro: Over 25 Years of Reliability Thanks to the B Formal Method*. Météor (Paris Métro Line 14) automatic train control software: over 110,000 lines of B model generating 86,000 lines of Ada; no bug found after proof completion through validation, integration, on-site testing, or 25+ years of revenue service. 2024. URL: https://www.clearsy.com/en/the-tools/extension-of-line-14-of-the-paris-metro-over-25-years-of-reliability-thanks-to-the-b-formal-method/ (visited on 08/30/2026).

[5] Chris Newcombe et al. "How Amazon Web Services Uses Formal Methods". In: *Communications of the ACM* 58.4 (2015). Primary account by AWS engineers: seven teams used TLA+ by 2015; TLA+ model checking found subtle DynamoDB design bugs missed by design review, code review, and testing. Executive management began proactively encouraging TLA+ specs afterward. URL: https://lamport.azurewebsites.net/tla/formal-methods-amazon.pdf.

[6] JetBrains. *Which AI Coding Tools Do Developers Actually Use at Work?* AI Pulse survey, 10,000+ professional developers, January 2026 wave: 90% use an AI tool at work; GitHub Copilot 29%, Cursor 18%, Claude Code 18% workplace adoption. 2026. URL: https://blog.jetbrains.com/research/2026/04/which-ai-coding-tools-do-developers-actually-use-at-work/ (visited on 08/30/2026).

[7] JetBrains. *AI Coding Agents: Adoption Trends*. May-July 2026 survey wave: Copilot workplace adoption falls to 21%, Cursor to 12%; 68% of developers use an AI agent daily; OpenCode reaches 7% adoption. 2026. URL: https://blog.jetbrains.com/research/2026/08/ai-coding-agent-adoption-2026/ (visited on 08/30/2026).

[8] Stack Overflow. *2025 Stack Overflow Developer Survey*. 49,009 responses, 166 countries, fielded May-June 2025. 84% use or plan to use AI tools (up from 76% in 2024); only 29% trust AI output to be accurate (down from 40% in 2024); 66% report AI solutions are frequently close but wrong. 2025. URL: https://survey.stackoverflow.co/2025/ai (visited on 08/30/2026).

[9] Sponsio. *Sponsio: Runtime Contract Enforcement for AI Agents*. Commercial product compiling natural-language policy into deterministic finite-state-machine (LTL) contracts checked at every agent tool call, with no LLM in the enforcement path. Closest existing product to a machine-decides-not-agent-decides architecture; formal layer is LTL/finite automata, not a full B-Method abstract-machine specification. 2026. URL: https://sponsio.dev/ (visited on 08/30/2026).

[10] Riddhi Mohan Sharma. *Ethical Hyper-Velocity (EHV): A Hardware-Rooted Zero-Trust Runtime Enforcement Architecture for Agentic AI Systems*. Sole-author preprint, not peer reviewed. TLA+ specification of a zero-trust agent runtime with a PERMIT/DENY/ESCALATE safety invariant; the TLC model check is bounded (1,738 states, 324 distinct, depth 8). 2026. URL: https://arxiv.org/abs/2605.17909.

[11] Thierry Lecomte et al. "Applying a Formal Method in Industry: A 25-Year Trajectory". In: *Formal Methods: Foundations and Applications (SBMF 2017)*. Vol. 10623. Lecture Notes in Computer Science. Retrospective on the B-Method's industrial trajectory since Météor Line 14, including B's adoption by Alstom's CBTC product line (roughly 100 metro lines worldwide). 2017, pp. 70–87. URL: https://arxiv.org/abs/2005.07190.

[12] Noble Saji Mathews and Meiyappan Nagappan. *Design Choices Made by LLM-Based Test Generators Prevent Them from Finding Bugs*. Evaluation of GitHub Copilot, Codium CoverAgent, and CoverUp on the Refactory dataset (2,442 correct and 1,783 buggy human-written Python programs); up to 68.1% of one tool's final generated test suites passed on incorrect implementations while failing on correct ones, with broader averages near 26–35% depending on whether the model saw buggy code. 2024. URL: https://arxiv.org/abs/2412.14137.

[13] METR. *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity*. Randomized controlled study: AI coding assistance made experienced developers roughly 19% slower on real tasks despite perceiving themselves as roughly 20% faster. 2025. URL: https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/.